GAME DESIGN DOCUMENT (GDD)

Eco-Scrapper: Reclamation

Restaure o ecossistema degradado através da automação, física e biotecnologia.

1. Visão Geral do Jogo

- Título do Jogo: Eco-Scrapper: Reclamation
- Subtítulo / Tagline: Restaure o ecossistema degradado através da automação, física e biotecnologia.
- Público-Alvo:
  - **Classificação indicativa:** Livre / 10+ anos (abrangendo crianças, adolescentes e jovens adultos).
  - Filosofia de Design: Curva de aprendizado acessível com controles intuitivos para o público mais jovem, combinada a desafios de física, puzzles espaciais e rotas de exploração interessantes para jogadores experientes.
  - Plataforma Alvo: PC / Windows, com suporte exclusivo para teclado e Mouse.
- Conceito do Jogo: Jogo de ação, plataforma e restauração ecológica em terceira pessoa 3D. O jogador controla Scrap-E (Unidade S-01), um carismático autômato bípede de limpeza que percorre vales devastados por acúmulo de lixo industrial e contaminação tóxica. Por meio de manipulação física do cenário, reciclagem de detritos e semeadura biológica, o jogador desobstrui rios soterrados e restaura a vegetação nativa setor por setor.
- Gênero: Aventura / Plataforma 3D / Puzzle Ambiental baseado em Física.
- Loop Principal de Gameplay (Core Loop) & Tríade de Puzzles:
  - Ritmo de Jogo: Exploração relaxada no ritmo do jogador, contando com um modo opcional de 'Eco-Timer' ativável no menu para quem busca desafios de velocidade.
  - Core Loop: Explorar o bioma → Identificar fontes de obstrução/poluição → Resolver puzzles físicos → Coletar bio-recursos → Reativar infraestrutura ecológica → Testemunhar a transformação ambiental.
  - As 3 Mecânicas Únicas de Puzzle:
    - 1. Puzzle de Impacto e Balística de Entulho (Mecânica AddForce/Torque): Desobstrução de calhas fluviais empurrando e rolando carcaças metálicas e pedras com aplicação de forças e rotação.
    - 2. Puzzle de Contrapeso e Balança de Sucata: Plataformas de gangorra mecânica onde o jogador empurra barris de massas distintas para erguer pontes e acessar platôs.
    - 3. Puzzle de Condução e Redirecionamento Bioativo: Posicionamento de condutores de sementes para guiar o fluxo hídrico recuperado até o núcleo da Torre Purificadora.

- Aparência e Estilo Visual (Art Direction):
  - Estilo Gráfico: 3D Low-Poly estilizado e vibrante via Universal Render Pipeline (URP).
  - Transformação Ambiental Dinâmica: O cenário reage em tempo real. No início, predominam tons ocres, névoa espessa e águas turvas; ao purificar o setor, a névoa se dissipa, a iluminação ganha calor dourado, o rio volta a fluir límpido e surgem brotos florais via partículas e shaders.
- Fluxo do Jogo e Navegação de Interfaces (UI Flow):
  - 1. Menu Principal: [Iniciar Jogo], [Instruções / Controles], [Opções (Toggle Eco-Timer)], [Sair].
  - 2. HUD Durante o Jogo: Contador dinâmico ('Bio-Filtros: X / 5'), status do ecossistema, prompt de interação e timer opcional.
  - 3. Menu de Pausa: [Continuar], [Reiniciar Nível] (botão com SceneManager), [Menu Principal].
  - 4. Telas de Vitória e Game Over: Estatísticas de purificação e botão funcional de reinício de cena.
  - O diagrama completo de relação entre as telas está na seção 3.9 (Fluxo de Telas).

2. História, Cenário e Personagens

2.1 História e Narrativa

- História de Fundo (Lore): Após décadas de exploração industrial sem controle na província de Aethelgard, complexos fabris foram abandonados, soterrando cursos d'água naturais sob toneladas de escória, sucata e tonéis de lodo químico. O solo tornou-se árido e a fauna foi expulsa pela névoa tóxica. Scrap-E (S-01), uma unidade autômata bípede de limpeza ecológica desativada há anos, é religada por um pulso residual de emergência do sistema ambiental central, guiado pela inteligência ARIA.
- Progressão da Narrativa:
  - *Ato 1 (Nível 1 - Vale Estéril):* Scrap-E desperta, restabelece contato com ARIA, desobstrui o canal da represa soterrada por sucatas e ativa a primeira Torre Purificadora, trazendo de volta o fluxo de água límpida e as primeiras espécies de vida animal.
  - Atos 2 a 5 (Visão Global): O avanço gradual para a Mina Química subterrânea, o Pântano Sintético, a Fábrica de Fusão e finalmente o Santuário da Biosfera, fechando o ciclo de reflorestamento total do vale.
- Cena de Diálogo / Script de Abertura:
  - *Atores:* Scrap-E (S-01) e ARIA (Inteligência Artificial de Gestão Ambiental).
  - *Cenário:* Plataforma de Despertar, cercada por fumaça cinzenta e carcaças de metal enferrujado.
  - *Script de Abertura (Caixas de Diálogo na UI com bips sintéticos):*
    - **[ARIA]:** "Sistemas vitais reengajados... Unidade Scrap-E S-01 operacional. Protocolo Aethelgard-Verde inicializado."

    - **[Scrap-E]:** *(Emite bipes sonoros mecânicos e projeta no visor um ícone de interrogação piscante).*
    - **[ARIA]:** "Alerta: O Vale Estéril atingiu índice crítico de contaminação por entulho. Nossos sensores detectam 5 Bio-Filtros dispersos no setor. Encontre-os, desobstrua o canal da represa usando propulsão de impacto e reinicie a Torre Purificadora."
    - **[Scrap-E]:** *(Exibe um check verde no visor e aciona as articulações dos braços com estalos metálicos).*

2.2 Lentes de Design

- Lente #1: Experiência Essencial:
    - *Experiência pretendida:* A gratificação visual e mecânica do renascimento ecológico. O contraste marcante entre o peso estéril do lixo metálico e a leveza orgânica da natureza brotando como resultado direto do trabalho físico do jogador.
    - Elementos essenciais: Peso e inércia realistas dos objetos ao serem empurrados; alívio auditivo da transição de ruídos industriais para som de água corrente; retorno visual de fauna e flora povoando a cena limpa.
    - Como o jogo captura a essência: Cada bloco retirado do leito d'água acelera a correnteza, e ao concluir a fase o cenário morto se transfigura em um oásis verdejante e vívido.
- Lente #2: Surpresa:
    - *O que vai surpreender os jogadores:*
        - *1. Desmoronamentos e Efeito Dominó:* O entulho empilhado não é estático; ao empurrar um bloco de suporte, a gravidade e o torque calculados pelo PhysX da Unity fazem a pilha ruir com efeito cascata, abrindo atalhos inesperados.
        - *2. Rejogabilidade por Spawn Procedural:* A posição dos Bio-Filtros e caixas de recursos muda a cada tentativa graças ao sorteio aleatório via vetor de prefabs (Random.Range), surpreendendo o jogador que reiniciar a fase.
        - *3. Aparição Súbita da Fauna Nativa:* Espécies de animais silvestres surgem e interagem com o cenário imediatamente após a névoa tóxica se dissipar.

2.3 Mundo do Jogo e Áreas do Nível 1

- Visual Geral e Atmosfera: Desfiladeiro rochoso de relevo acidentado cortado por um leito de rio seco e lixões industriais. O ar começa carregado com névoa volumétrica acinzentada que é gradualmente substituída por um horizonte límpido e céu aberto.
- Setorização do Nível 1:
    a. *1. Plataforma de Despertar:* Ponto de spawn elevado e seguro. Introdução aos controles básicos de caminhada, salto e câmera.
    b. 2. Ravina Entulhada: Gargalo estreito bloqueado por contêineres e pedras que exige o primeiro puzzle de colisão física e força (AddForce) para abrir passagem.
    c. 3. Leito Fluvial Seco: Área central ampla com poças tóxicas (Tag: ToxicSlime), barris de contrapeso para a gangorra e os pontos de spawn randômico dos Bio-Filtros.

d. 4. Base da Represa e Torre Purificadora: O ponto culminante do nível, contendo a comporta travada que necessita de rotação/torque e o pedestal de ativação ecológica.

2.4 Personagens e Entidades Vivas

- Scrap-E (Unidade S-01) - Protagonista Jogável:
  - *Personalidade:* Curioso, ágil, expressivo e incansável em sua diretriz ecológica.
  - Aparência: Autômato compacto bípede estilizado em Low-Poly, carcaça amarela com marcas de desgaste, olhos/visor em display LED azul que reagem com emoticons às ações do jogador.
  - Animações Mixamo Integradas: Idle (respiração mecânica), Walk (passos firmes), Run (corrida acelerada), Jump (impulso vertical com agachamento prévio) e Fall (braços abertos em desaceleração terminal).
  - Habilidades: Movimentação ágil, salto, aplicação de impacto direcional (AddForce) e torque de tombamento em objetos físicos.
- ARIA (Voz Guia / NPC de Suporte):
  - Sistema de inteligência artificial remota que atua via rádio do HUD, emitindo objetivos de missão, avisos de perigo e confirmações de purificação.
- Fauna Reativa do Vale (Animais Nativos que contabilizam na meta de assets):
  - Lagarto das Rochas (Sand-Gecko): Réptil de baixa poligonagem que corre pelas pedras assim que a temperatura do solo normaliza.
  - Pássaro de Encosta (Canyon Swift): Pequenas aves que pousam nas tubulações limpas e voam em bandos assim que a torre dispersa a fumaça.
  - Lebre do Prado (Eco-Hare): Pequeno mamífero herbívoro que surge saltando pelas novas moitas de capim que nascem ao redor da água límpida.

3. Jogabilidade e Mecânicas

3.1 Gameplay, Objetivos e Condições de Vitória/Derrota

- Objetivo Principal do Nível 1: Purificar a nascente do Vale Estéril coletando os 5 Bio-Filtros dispersos, removendo os bloqueios físicos que estrangulam o canal da represa e acionando o console central da Torre Purificadora.
- Estrutura das Missões e Enigmas:
  a. *Fase de Exploração & Coleta:* Rastrear o vale para coletar os 5 Bio-Filtros bioativos gerados proceduralmente (via vetor de prefabs aleatorizado com Random.Range).
  b. Fase de Limpeza Física (Puzzle AddForce/Torque): Desobstruir a calha d'água empurrando 3 blocos de carcaça industrial e barris que bloqueiam o leito. O jogador pode esbarrar neles para um deslocamento suave ou desferir o Pulso Cinético (Botão Esquerdo / tecla E) para arremessar e rolar as peças com alto impacto de AddForce e torque angular.
  c. Fase de Ativação do Purificador: Depositar os 5 filtros no receptor e interagir com o console da represa para abrir o fluxo hídrico e detonar a onda de restauração do

bioma.

- Condições de Vitória:
  - Inserção dos 5 Bio-Filtros na torre + remoção dos 3 entulhos obstrutores + acionamento da torre.
  - Feedback de Vitória: Abertura do fluxo de água cristalina, explosão esférica de partículas de pólen/sementes, clareamento da névoa cinzenta e exibição da Tela de Vitória com pontuação final e tempo registrado.
- Condições de Derrota (Game Over):
  - *Dano Crítico por Lodo Tóxico:* Cair ou permanecer nas poças químicas cáusticas (Tag: `ToxicSlime`) ou ser esmagado por detritos rochosos em queda livre zera a barra de Integridade Estrutural (100 HP).
  - *Esgotamento do Eco-Timer (Modo Opcional):* Caso o jogador ative o cronômetro no menu de opções, o tempo limite do Nível 1 é de 3 minutos. Se o relógio zerar antes da purificação, o nível é considerado falho por sobrecarga de contaminação.
  - *Feedback de Derrota:* Scrap-E entra em curto-circuito (faíscas/fumaça), e surge a Tela de Game Over contendo o botão obrigatório de **Reiniciar Jogo** (`SceneManager.LoadScene`).

3.2 Mecânicas Técnicas Obrigatórias (1º Bimestre)

- **Movimentação do Jogador:**
  - Controle híbrido de precisão combinando `Transform.Translate` para ajustes de orientação e micro-posicionamento junto a `Rigidbody.AddForce` para aceleração contínua, inércia de corrida e impulso vetorial de salto.
- Interação com a Física em Duplo Estágio (AddForce e Torque):
  - *Estágio 1 - Empurrão Suave por Contato:* Ao caminhar e colidir diretamente com objetos contendo `Rigidbody` (caixotes leves e barris), Scrap-E transfere força suave proporcional à sua velocidade de deslocamento.
  - *Estágio 2 - Pulso Cinético Direcional (Ação Dinâmica):* Ao pressionar o **Botão Esquerdo do Mouse** ou tecla **E**, Scrap-E libera uma descarga propulsora concentrada nos braços. O script detecta colliders frontais e aplica uma força impulsiva massiva (`AddForce(direcao * forca, ForceMode.Impulse)`) combinada a um torque angular rotacional (`AddTorque(eixo * torque, ForceMode.Impulse)`), fazendo caçambas, pedras e carcaças rolarem e tombarem com balística realista.
- **Rotação Direta de Objetos (Transform.Rotate):** Diferente do torque, que gira objetos pela física, a rotação é aplicada diretamente pelo script, sem depender de forças.
  - Bio-Filtros: giram continuamente no próprio eixo Y (`transform.Rotate(Vector3.up * velocidade * Time.deltaTime)`), destacando-se no cenário para indicar que são coletáveis.
  - Válvula da Comporta (asset 57): gira enquanto o jogador mantém a tecla E pressionada ao lado dela; ao completar a volta, a comporta da represa é destravada.
  - Orientação do Scrap-E: o robô gira suavemente para a direção em que está se movendo, acompanhando a câmera.

- Vetor de Prefabs e Spawn Aleatório:
  - Script gerenciador central (`SpawnManager.cs`) contendo vetores serializados: `GameObject[] itemPrefabs` e `Transform[] spawnPoints`.
  - Uma rotina seleciona dinamicamente índices não repetidos via `Random.Range(0, spawnPoints.Length)`, instanciando os 5 Bio-Filtros e os contêineres de sucata em locais distintos a cada reinício de fase, garantindo rejogabilidade e dinamismo.
- Sistema de Tags Organizado:
  - `Player`: Atribuída ao Scrap-E para detecção de gatilhos ambientais.
  - EcoItem: Bio-Filtros essenciais e sementes primárias.
  - ScrapCoin: Moedas/sucatas metálicas recicláveis opcionais.
  - HeavyDebris: Entulhos industriais e pedras rígidas com física ativa.
  - ToxicSlime: Poças de lodo contaminado que aplicam dano contínuo.
  - PurifierTrigger: Volume de interação com o console da torre central.
- **Efeitos de Partículas (Shuriken Particle System):**
  - Fumaça e Fuligem: Emissão constante de fumaça cinzenta opaca saindo de drenos rompidos e das margens de lodo químico.
  - Impacto e Pedrinhas: Disparo direcional de lascas de pedra e faíscas metálicas gerado exatamente no ponto de colisão do Pulso Cinético com detritos rígidos.
  - Onda de Purificação (Explosão Ecológica): Disparo esférico expansivo de partículas verdejantes brilhantes e pétalas ao religar a torre, seguido pelo desvanecimento suave do sistema de névoa.
- Interface do Usuário (UI / Canvas URP):
  - *HUD em Tempo Real:* Contador dinâmico *"Bio-Filtros: X / 5"*, contador de *"Sucata Reciclada: Y peças"*, barra visual de Integridade do Robô (HP) e cronômetro decrescente (visível apenas no modo Eco-Timer).
  - Telas de Transição: Menus de Vitória e Game Over com exibição de pontuação final calculada e botão funcional de Reiniciar Nível.

3.3 Economia do Jogo e Sistema de Pontuação (Score Final)

- Bio-Filtros (Recurso Vital): 5 unidades obrigatórias para progressão. Não possuem valor de troca, são a chave de reativação ambiental.
- Sucata Metálica Reciclável (Moeda de Pontuação):
  - Dispersas pelo vale e liberadas ao empurrar ou destruir contêineres velhos de lixo.
  - Cálculo de Pontuação Final (Score):
    - Cada peça de sucata reciclada coletada = +100 Pontos.
    - Conclusão do Nível 1 = +1.000 Pontos base.
    - Bônus de Tempo (Modo Eco-Timer): +10 pontos para cada segundo restante no cronômetro.
    - A pontuação final é consolidada na Tela de Vitória, criando incentivo para replay e rotas otimizadas de coleta. No segundo bimestre, essas moedas servirão para melhorias de velocidade e upgrades em módulos de habilidades.

3.4 Macetes, Easter Eggs e Interações Divertidas

- Easter Egg WALL-E & EVA: Escondido em uma caverna lateral atrás de um deslizamento de pedras quebráveis, o jogador encontra uma esteira de lagarta enferrujada e um extintor de incêndio vintage repousando ao lado de uma pequena bota velha contendo um broto verde saudável, homenageando os robôs clássicos da animação.
- Botão de Emote / Comportamento do Scrap-E:
  - Tecla G: Scrap-E para e aciona uma animação cômica de dança robótica/comemoração, disparando bipes afinados e um ícone de sorriso luminoso no seu visor LED.
  - Tecla H: Scrap-E solta uma gargalhada metálica sintetizada com pequenas faíscas inofensivas saindo de sua antena.
- Trapaça de Desenvolvedor (Cheat Code): Ao pressionar a sequência de teclas E-C-O no teclado durante a partida, o jogo executa um comando de depuração: recarrega a vida ao máximo e concede temporariamente um impulso de velocidade duplicada (Speed Boost) para facilitar a validação e demonstração dos professores em bancada.

3.5 Tabela dos 5 Elementos Temáticos de Ecorenovação (1º Bimestre)

Elemento Temático	Classificação	Papel no Gameplay e na Ecorenovação
Bio-Filtros de Solo	Coletável Essencial	Dispositivos purificadores que convertem terra estéril e viabilizam o reinício da torre.
Torre Purificadora	Infraestrutura	Estrutura vertical bioenergética que dissipa a névoa ácida e bombeia água limpa.
Mudas e Grama Bioativa	Entidade Viva	Flora que brota processualmente do solo à medida que o canal é desobstruído.
Entulho e Sucatas Metálicas	Física / Recurso	Barreiras físicas que sufocam o leito do rio e rendem pontos/moedas ao serem recicladas.
Calha Fluvial Despoluída	Dinâmica Ambiental	O curso d'água represado que volta a correr límpido e restabelece a bacia ecológica.

3.6 Combate e Conflito

- O jogo **não possui combate direto** no Nível 1. Não há inimigos armados e o Pulso Cinético não é uma arma: ele afeta apenas objetos físicos com a Tag `HeavyDebris` e não causa dano

à fauna.

- O conflito é **ambiental**: o jogador enfrenta o próprio cenário contaminado, modelado da seguinte forma:
  - Lodo Tóxico (`ToxicSlime`): enquanto o Scrap-E estiver em contato com a poça, perde 10 HP por segundo. O dano é detectado por colisão/trigger com a Tag e aplicado de forma contínua.
  - Detritos em Queda: entulhos que caem sobre o Scrap-E após um desmoronamento causam 25 HP de dano por impacto, apenas quando a velocidade do objeto no momento da colisão for alta (queda livre), evitando dano ao simplesmente esbarrar em uma pedra parada.
  - Pressão do Tempo (opcional): no modo Eco-Timer, o cronômetro funciona como um adversário indireto, forçando rotas mais rápidas.
- Recuperação: a Integridade Estrutural não se regenera sozinha no Nível 1; o jogador precisa evitar os perigos. O cheat E-C-O restaura o HP para demonstração.

3.7 Opções do Jogo

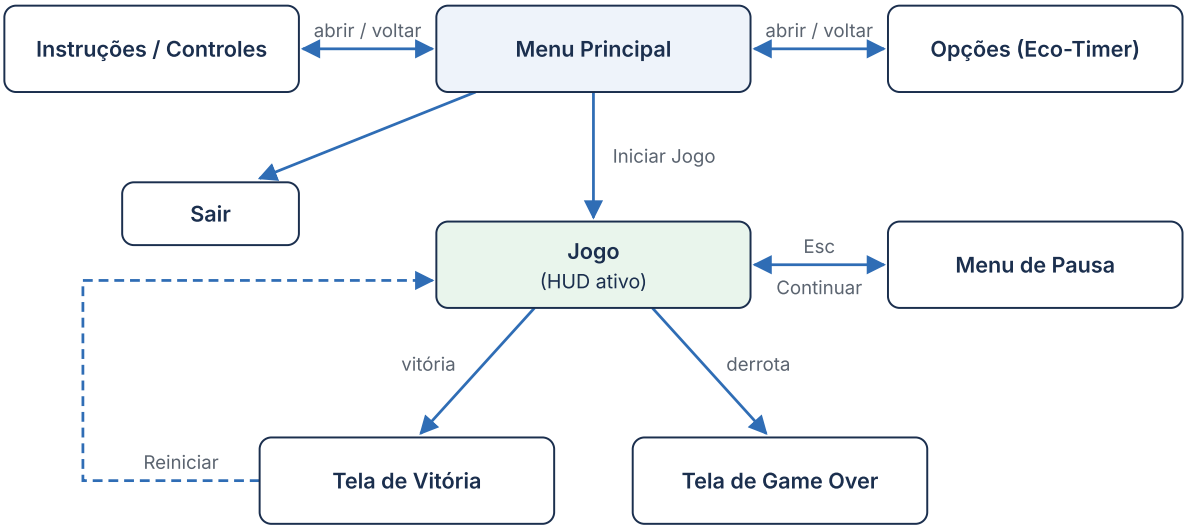
- **Eco-Timer (Ligado / Desligado):** acessível pelo menu [Opções] da tela inicial. Desligado por padrão.
  - Desligado: exploração livre e sem limite de tempo; o cronômetro não aparece no HUD e não há derrota por tempo.
  - Ligado: o cronômetro decrescente de 3 minutos aparece no topo da tela, o esgotamento do tempo leva ao Game Over e cada segundo restante na vitória vale +10 pontos.
- A escolha é mantida durante toda a sessão de jogo, inclusive ao reiniciar o nível.

3.8 Replay e Salvamento

- **Salvamento:** o Nível 1 é uma fase curta e contínua (aproximadamente 5 a 8 minutos), por isso **não possui sistema de save**. Cada partida começa do início do nível.
- **Replay:** o nível pode ser reiniciado a qualquer momento pelo botão [Reiniciar] das telas de Pausa, Vitória e Game Over, ou pela tecla R. O reinício recarrega a cena com `SceneManager.LoadScene`.
- **Incentivo à rejogabilidade:** a cada reinício, o spawn aleatório muda a posição dos Bio-Filtros e das sucatas, e a pontuação final estimula o jogador a buscar rotas melhores e mais sucata.
- **Planejado para o 2º bimestre:** salvamento do progresso entre níveis e das melhorias compradas com sucata, utilizando `PlayerPrefs` (armazenamento local da Unity).

3.9 Fluxo de Telas

O diagrama abaixo mostra como cada tela se relaciona com as outras. As setas indicam a ação do jogador que leva de uma tela à outra.



Pausa, Vitória e Game Over têm os botões [Reiniciar], que recarrega a cena do Jogo, e [Menu Principal], que volta ao Menu Principal.

- **Menu Principal:** ponto de entrada do jogo. Leva ao Jogo, às Instruções, às Opções ou fecha a aplicação.
- **Instruções / Controles:** mostra o mapeamento de teclas (seção 5.3) e o objetivo do nível. Botão [Voltar] retorna ao Menu.
- **Opções:** liga ou desliga o Eco-Timer (seção 3.7). Botão [Voltar] retorna ao Menu.
- **Jogo (HUD):** tela principal da partida, com HP, contador de Bio-Filtros, contador de sucata e cronômetro opcional.
- **Menu de Pausa:** aberto com Esc, congela o jogo. Permite continuar, reiniciar o nível ou voltar ao Menu Principal.
- **Vitória / Game Over:** exibem a pontuação final e o tempo. Ambas têm o botão obrigatório [Reiniciar], que recarrega a cena do Jogo, e o botão [Menu Principal].

4. Planejamento dos Níveis

4.1 Nível 1: O Vale Estéril (Foco Integral do 1º Bimestre)

- Sinopse do Nível: Desfiladeiro árido e pedregoso cortado por um leito de rio seco e soterrado por detritos industriais. O jogador aprende os controles de forma orgânica, desobstruir as calhas d'água usando física impulsiva (AddForce/Torque), coleta os 5 Bio-Filtros dispersos e reativa a Torre Purificadora central, transformando o bioma poluído em um oásis vívido.
- Tutorial de Treinamento Integrado (Primeiros 60 Segundos):
  - Ambientado diretamente na *Plataforma de Despertar*.
  - Passo 1 (Locomoção): O jogador movimenta Scrap-E usando W, A, S, D e ajusta a câmera em terceira pessoa com o mouse.
  - Passo 2 (Salto): Um desnível simples exige o uso da barra de espaço para saltar em um platô elevado.

    - Passo 3 (Pulso Físico): Uma barricada de caixotes leves bloqueia a saída da clareira; o prompt na tela ensina o Pulso Cinético (Botão Esquerdo / tecla E) para arremessar e rolar as peças com física ativa.
    - Passo 4 (Coleta): O primeiro Bio-Filtro reluz logo à frente, ensinando a colisão/coleta por aproximação antes da entrada na Ravina Principal.
- Catalogação Detalhada dos 70+ Assets do Nível 1 (Por Categorias):
    - Categoria 1: Terreno, Geologia e Formações Rochosas (20 Assets)
        - 1. Rocha Modular de Falésia Grande A
        - 2. Rocha Modular de Falésia Grande B
        - 3. Pedregulho Médio Pontiagudo (com colisor físico)
        - 4. Pedregulho Médio Arredondado
        - 5. Laje de Pedra Chata (Plataforma natural)
        - 6. Conjunto de Seixos e Lascas de Pedra Pequenas
        - 7. Arco Rochoso Natural de Entrada
        - 8. Paredão de Cânion Desgastado
        - 9. Coluna de Rocha Basáltica
        - 10. Bloco Rochoso Quebrável de Atalho
        - 11. Rampa de Terra Batida
        - 12. Talude de Cascalho Seco
        - 13. Textura de Solo Árido Fendido (Material PBR)
        - 14. Textura de Cascalho e Areia Grossa
        - 15. Textura de Rocha Sedimentar
        - 16. Textura de Lodo Tóxico Químico (Emissivo)
        - 17. Textura de Terra Fértil e Gramado (Pós-Restauração)
        - 18. Malha de Calha Fluvial Seca
        - 19. Margem de Seixos de Rio
        - 20. Crateras de Erosão do Solo
    - Categoria 2: Flora Árida e Bioativa (15 Assets)
        - 21. Árvore Reta Petrificada e Seca
        - 22. Árvore Retorcida Caída (Obstáculo transponível)
        - 23. Toco de Tronco Oco Apodrecido
        - 24. Raiz Gigante Exposta e Ressecada
        - 25. Tufo de Arbusto Seco Espinhoso
        - 26. Ramo de Espinheiro Rasteiro
        - 27. Arbusto Queimado por Ácido
        - 28. Muda Bioativa de Caule Bioluminescente (Estágio Broto)
        - 29. Flor Purificadora de Esporos Azuis (Surgimento dinâmico)
        - 30. Moita de Grama Verdejante Alta (Pós-Restauração)
        - 31. Conjunto de Trepadeiras Vivas de Encosta
        - 32. Cogumelo Filtrador de Solo

        - 33. Líquen Musgoso Fluorescente
        - 34. Aglomerado de Juncos Aquáticos
        - 35. Arbusto Frutífero Restaurador
    - Categoria 3: Resíduos Industriais e Sucatas com Física (18 Assets)
        - 36. Tambor de Óleo Amassado (Rigidbody - Massa 15kg)
        - 37. Barril Químico de Contrapeso (Rigidbody - Massa 50kg)
        - 38. Caçamba de Metal Enferrujada Grande
        - 39. Duto Metálico Cilíndrico Quebrado A (Pedaço Curvo)
        - 40. Duto Metálico Quebrado B (Pedaço Reto transponível)
        - 41. Caixa Industrial de Madeira Compensada (Quebrável)
        - 42. Caixote de Aço Reforçado de Bloqueio
        - 43. Viga Estrutural I-Beam Torcida
        - 44. Engrenagem Mecânica Desgastada de 1.5m
        - 45. Eixo de Roda Dentada Pequena
        - 46. Rolo de Cabos Elétricos de Cobre Desencapados
        - 47. Pneu Industrial Tratorado Descartado
        - 48. Placa Metálica Corrugada de Entulho
        - 49. Chapa de Aço para Gangorra de Contrapeso
        - 50. Carcaça de Gerador a Óleo Desativado
        - 51. Tanque de Pressão Esférico Furado
        - 52. Pedaço de Escada de Incêndio Torcida
        - 53. Sucata Metálica Reciclável (Coletável ScrapCoin)
    - Categoria 4: Infraestrutura, Hidráulica e Mecanismos (10 Assets)
        - 54. Torre Purificadora Central (Estrutura Bioenergética)
        - 55. Console / Terminal Interativo da Torre
        - 56. Comporta Hidráulica de Concreto da Represa
        - 57. Roda / Válvula Mecânica de Rotação Manual
        - 58. Passarela de Grade de Aço Industrial
        - 59. Pilar de Sustentação de Concreto Armado
        - 60. Corrimão Metálico de Proteção Desgastado
        - 61. Tubo Central de Drenagem Pluvial
        - 62. Caixa de Distribuição Elétrica Quebrada
        - 63. Poste de Sinalização Industrial Inclinado
    - Categoria 5: Personagens, Entidades Vivas e Fauna em Destaque (9 Assets)
        - 64. Protagonista Scrap-E (Unidade S-01 com Rig Humanoide)
        - 65. Terminal Holográfico / Drone ARIA
        - 66. Lagarto das Rochas (Sand-Gecko): Réptil veloz que toma sol nas lajes aquecidas e reage à aproximação.
        - 67. Pássaro Canyon Swift: Pássaro de encosta que pousa nos cabos e sobrevoa o cânion após a fumaça sumir.

        - 68. Lebre do Prado (Eco-Hare): Mamífero ágil que salta entre os novos arbustos verdes restaurados.
        - 69. Besouro-Blindado de Carapaça Metálica: Inseto bio-mecânico que se alimenta de resíduos de ferrugem perto da água.
        - 70. Peixe de Água Doce Cintilante: Nada nas calhas limpas assim que o fluxo da represa é restabelecido.
        - 71. Easter Egg - Esteira de Lagarta e Bota com Broto (Homenagem WALL-E).
        - 72. Easter Egg - Extintor de Incêndio Vintage Enferrujado.

4.2 Visão Geral dos Níveis 2 a 5 (Planejamento Global do Jogo)

- Nível 2: A Mina de Lodo Químico (2º Bimestre)
    - Ambiente: Complexo de escavação subterrâneo com túneis escuros, poços de piche borbulhante e trilhos desmoronados.
    - Desafio Chave: Deslocamento vertical através de plataformas móveis movidas por peso e resolução de puzzles de vazamento de gás usando válvulas giratórias de torque.
- Nível 3: O Pântano Sintético (2º Bimestre)
    - Ambiente: Zonas alagadas rasas com contaminação eletrolítica e vegetação artificial apodrecida.
    - Desafio Chave: Uso de NavMesh para navegação tática, desvio de drones de vigilância corrompidos e condução de correntes elétricas purificadoras pela água.
- Nível 4: A Fábrica Abandonada de Fusão (2º Bimestre)
    - Ambiente: Fábrica automatizada colossal com esteiras ativas, braços robóticos descalibrados e fornos industriais.
    - Desafio Chave: Puzzles de temporização, uso intensivo de AddForce para lançar baterias de fusão em trilhos elevados e ativação de esteiras por torque mecânico.
- Nível 5: O Santuário da Biosfera (2º Bimestre)
    - Ambiente: Cúpula geodésica colossal onde convergem os cursos d'água de toda a província, tomada por raízes sintéticas e névoa residual.
    - Desafio Chave: Encontro climático integrando os três temas: Ecorenovação (semeadura final), Tecnologia (religar o reator central da ilha) e Skills (parkour de alta precisão entre pontes colapsadas).

5. Interface, Controles e Áudio

5.1 Sistema Visual e HUD (Heads-Up Display)

- Estilo da Câmera: Visão em terceira pessoa (perspectiva) posicionada a meia distância sobre o ombro do autômato, com amortecimento cinemático suave via Cinemachine / interpolação vetorial.
- Elementos Visuais do HUD (Canvas URP):

- *Barra de Integridade Estrutural (HP):* Localizada no canto superior esquerdo com medidor de integridade (0 a 100 HP) e contorno metálico estilizado.
- Contador de Bio-Filtros (Objetivo Principal): Ícone orgânico com texto dinâmico "Bio-Filtros: X / 5" com pulso luminoso ao coletar cada unidade.
- Contador de Sucata Reciclada (Score Secundário): Ícone de engrenagem/moeda metálica "Sucata: Y peças (+Score)".
- Cronômetro Decrescente (Eco-Timer): Exibido no topo central da tela apenas quando o modo contrarrelógio for ativado no menu de opções.

5.2 Sistema de Ajuda e Pop-ups Contextuais

- Prompts Dinâmicos em Tela: Notificações flutuantes limpas e não intrusivas que surgem próximo à mira quando o Scrap-E se aproxima de elementos interativos:
  - Próximo a barricadas de entulho com física: [Botão Esquerdo / E] Disparar Pulso Cinético
  - Próximo ao console da Torre Purificadora: `[E] Depositar Filtros e Iniciar Purificação`
  - Próximo a desníveis e fendas rochosas: `[Espaço] Saltar`

5.3 Sistema de Controle (Mapeamento Completo PC)

- `W, A, S, D`: Movimentação omnidirecional do robô (frente, ré, esquerda, direita).
- `Mouse (Eixo X/Y)`: Controle de órbita da câmera e mira direcional.
- `Barra de Espaço`: Salto com impulso físico vertical via Rigidbody.
- `Botão Esquerdo do Mouse / E`: Disparo do Pulso Cinético direcional de impacto com física ativa.
- `G`: Emote 1 (Dança robótica comemorativa no visor LED).
- `H`: Emote 2 (Gargalhada metálica com faíscas cômicas).
- `R`: Reinício instantâneo do nível (atalho de debug/praticidade).
- `Esc`: Abertura do Menu de Pausa.
- `E-C-O (Sequência de Teclas)`: Cheat code de depuração para bancada (regenera vida e ativa impulso de velocidade).

5.4 Áudio e Feedback Sonoro

- **Locomoção e Ações do Protagonista:** Ruído mecânico rítmico de servomotores nos passos de Scrap-E; cliques e zumbidos ao acionar os propulsores de salto.
- **Física e Colisões:** Efeitos sonoros de choque metálico pesado e estalos ocos ao arremessar barris, engrenagens e vigas com AddForce.
- **Áudio Ambiente Dinâmico:**
  - *Fase Contaminada:* Zumbido estático grave, vento desértico e gotejar químico corrosivo.
  - *Fase Purificada:* Dissipação imediata dos sons industriais, dando lugar a acordes acústicos serenos e ao murmúrio de água cristalina fluindo.

6. Documentação Técnica do Código

Seguindo a norma de avaliação da disciplina, toda função dos scripts do projeto possui um cabeçalho de documentação no formato XML do C# (`/// <summary>`), descrevendo:

- **<summary>**: o que a função faz;
- **<param>**: o que representa cada argumento de entrada;
- **<returns>**: o que a função retorna (ou que não há retorno);
- **<remarks>**: o passo a passo do que é feito no corpo da função.

Os scripts ficam em `Assets/Scripts` e foram escritos no estilo dos tutoriais da disciplina (variáveis públicas configuráveis no Inspector e lógica nos métodos `Update`, `FixedUpdate` e nos eventos de colisão). O projeto também contém o utilitário de editor `MixamoSetupUtility.cs` (em `Assets/Art/Personagem/Editor`), que automatiza a importação de animações do Mixamo; por ser uma ferramenta de desenvolvimento, que não roda durante o jogo, ele não está reproduzido aqui.

6.1 Visão geral dos scripts

Script	Autor	Função no jogo	Critérios da disciplina atendidos
`PlayerMovement.cs`	Thiago	Movimentação do Scrap-E: andar e correr com `Transform.Translate`, girar com `Transform.Rotate`, pular com `Rigidbody.AddForce`, detecção de chão por colisão e envio dos parâmetros ao Animator.	Movimentação (Translate e AddForce), rotação, colisão, animação
`InteractionController.cs`	Thiago	Pulso Cinético: detecta os entulhos à frente com `Physics.OverlapSphere`, filtra pela tag `HeavyDebris` e arremessa com `AddForce` e `AddTorque`; cria a partícula de impacto por função.	AddForce, torque, tag, partículas, spawn por função, som
`SpawnManager.cs`	Thiago	Cria os Bio-Filtros e as sucatas a partir de vetores de prefabs, sorteando pontos de spawn com `Random.Range` sem repetição.	Vetor de prefabs, randomizar a posição no vetor, spawn de objetos por função
`Rotator.cs`	Thiago	Gira continuamente os itens coletáveis no próprio eixo.	Rotação (`Transform.Rotate`)
`PlayerInventory.cs`	Thiago	Coleta de Bio-Filtros e sucatas por trigger, identificando cada item pela tag.	Colisão (trigger), tag, som

`PlayerHealth.cs`	Thiago	Integridade estrutural (HP): dano contínuo do lodo tóxico, dano de detritos em movimento e Game Over.	Colisão, tag
`PurifierTower.cs`	Thiago	Torre Purificadora: verifica as condições da fase, dispara a explosão de partículas e o som, dissipa a névoa e declara a vitória.	Colisão (trigger), tag, partículas, som, spawn por função
`GameManager.cs`	Thiago	Estado da fase: pausa, Eco-Timer, vitória, derrota e cálculo da pontuação.	Lógica de jogo, UI
`UIManager.cs`	Thiago	Atualiza o HUD (HP, filtros, sucata, cronômetro) e exibe os painéis de pausa, vitória e Game Over.	UI (texto e botão)
`CheatCode.cs`	Thiago	Trapaça de desenvolvedor E-C-O (GDD 3.4): recupera a vida e dobra a velocidade por alguns segundos, para facilitar a demonstração em bancada.	Macetes e Easter Eggs (template do GDD)
`MainMenuController.cs`	Rafael	Menu Principal, vitrine 3D do Scrap-E e botões de navegação, incluindo o botão obrigatório de reiniciar o jogo.	UI (texto e botão), botão de reiniciar, rotação

6.2 Script: PlayerMovement.cs

Movimentação do Scrap-E: andar e correr com `Transform.Translate`, girar com `Transform.Rotate`, pular com `Rigidbody.AddForce`, detecção de chão por colisão e envio dos parâmetros ao Animator. *Autor: Thiago.*

```
using UnityEngine;

/// <summary>
/// Controla a movimentação do Scrap-E: andar, correr, girar e pular.
/// </summary>
public class PlayerMovement : MonoBehaviour
{
    public float speed = 5f;
    public float runSpeed = 8f;
    public float turnSpeed = 150f;
    public float jumpForce = 6f;
    public Animator animator;

    private Rigidbody playerRb;
    private bool isOnGround = true;

    /// <summary>
    /// Pega o Rigidbody do jogador quando o jogo começa.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
```

```
    /// <remarks>
    /// Guarda a referência ao Rigidbody e trava a rotação física,
    /// para o robô não tombar ao bater em algo.
    /// </remarks>
    void Start()
    {
        playerRb = GetComponent<Rigidbody>();
        playerRb.freezeRotation = true;
    }

    /// <summary>
    /// Lê o teclado a cada frame e move, gira e faz o jogador pular.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. W/S movem para frente e para trás com Translate (Shift corre).
    /// 2. A/D giram o robô com Rotate.
    /// 3. Espaço aplica AddForce para cima, se estiver no chão.
    /// 4. Envia a velocidade e o estado do chão para o Animator.
    /// </remarks>
    void Update()
    {
        float horizontalInput = Input.GetAxis("Horizontal");
        float verticalInput = Input.GetAxis("Vertical");
        bool isRunning = Input.GetKey(KeyCode.LeftShift);
        float currentSpeed = isRunning ? runSpeed : speed;

        transform.Translate(Vector3.forward * Time.deltaTime * currentSpeed * verticalInput);
        transform.Rotate(Vector3.up, Time.deltaTime * turnSpeed * horizontalInput);

        if (Input.GetKeyDown(KeyCode.Space) && isOnGround)
        {
            playerRb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
            isOnGround = false;

            if (animator != null)
            {
                animator.SetTrigger("Jump");
            }
        }

        if (animator != null)
        {
            float animSpeed = Mathf.Abs(verticalInput) * (isRunning ? 1f : 0.5f);
            animator.SetFloat("Speed", animSpeed);
            animator.SetBool("IsGrounded", isOnGround);
        }
    }

    /// <summary>
    /// Remove qualquer velocidade horizontal que a física tenha dado ao jogador.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// O movimento horizontal é feito só pelo Translate. Quando o jogador encosta
    /// numa parede ou objeto, a física o empurra para fora e isso gera velocidade
    /// no Rigidbody. Como o collider não tem atrito, essa velocidade nunca pararia
    /// e o robô deslizaria sozinho. Aqui zeramos X e Z e mantemos Y, que é a
    /// gravidade e o pulo.
    /// </remarks>
    void FixedUpdate()
    {
        Vector3 velocity = playerRb.linearVelocity;
        playerRb.linearVelocity = new Vector3(0f, velocity.y, 0f);
    }

    /// <summary>
    /// Verifica, a cada passo de física, se o jogador está apoiado em alguma superfície.
```

```
    /// </summary>
    /// <param name="collision">Dados da colisão, incluindo todos os pontos de contato.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Roda enquanto o contato durar (não só quando ele começa), então funciona
    /// mesmo se o jogador nunca chegou a se separar do chão.
    /// 1. Ignora enquanto o jogador ainda está subindo, para não "recarregar" o pulo no ar.
    /// 2. Percorre os pontos de contato: se algum tem a normal apontando para cima
    ///    (a superfície está embaixo do jogador), considera que ele está no chão.
    ///    Contatos com paredes (normal para o lado) são ignorados.
    /// </remarks>
    void OnCollisionStay(Collision collision)
    {
        if (playerRb.linearVelocity.y > 0.1f)
        {
            return;
        }

        foreach (ContactPoint contact in collision.contacts)
        {
            if (contact.normal.y > 0.5f)
            {
                isOnGround = true;
                return;
            }
        }
    }
}
```

6.3 Script: InteractionController.cs

Pulso Cinético: detecta os entulhos à frente com `Physics.OverlapSphere`, filtra pela tag `HeavyDebris` e arremessa com `AddForce` e `AddTorque`; cria a partícula de impacto por função. *Autor: Thiago.*

```
using UnityEngine;

/// <summary>
/// Controla o Pulso Cinético do Scrap-E: ao apertar E ou o botão esquerdo do mouse,
/// arremessa os entulhos (tag HeavyDebris) que estão na frente do robô, usando
/// AddForce (empurrão) e AddTorque (giro).
/// </summary>
public class InteractionController : MonoBehaviour
{
    public float pulseForce = 12f;          // força do arremesso
    public float pulseTorque = 8f;          // força do giro aplicado no objeto
    public float pulseRadius = 2f;          // tamanho da área atingida
    public float pulseDistance = 1.5f;      // distância à frente do robô onde fica o centro da
área
    public float cooldown = 0.5f;           // tempo mínimo entre dois pulsos (segundos)
    public GameObject impactParticlePrefab; // partícula de pedrinhas (prefab do Arthur, opcional)
    public AudioClip pulseSound;            // som do pulso (opcional)

    private float nextPulseTime = 0f;

    /// <summary>
    /// Verifica a cada frame se o jogador apertou o botão do pulso.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Se E ou o botão esquerdo do mouse foi apertado e o tempo de recarga
    /// já passou, dispara o pulso e marca quando o próximo será permitido.
    /// </remarks>
    void Update()
```

```
    {
        bool pressedPulse = Input.GetKeyDown(KeyCode.E) || Input.GetMouseButtonDown(0);

        if (pressedPulse && Time.time >= nextPulseTime)
        {
            FireKineticPulse();
            nextPulseTime = Time.time + cooldown;
        }
    }

    /// <summary>
    /// Dispara o Pulso Cinético e arremessa os entulhos próximos.
    /// </summary>
    /// <returns>int: quantidade de objetos atingidos pelo pulso.</returns>
    /// <remarks>
    /// 1. Calcula o centro da área do pulso, um pouco à frente do robô.
    /// 2. Usa Physics.OverlapSphere para listar todos os colliders dentro dessa esfera.
    /// 3. Para cada um com a tag HeavyDebris e com Rigidbody:
    ///    - calcula a direção do robô até o objeto, inclinada um pouco para cima;
    ///    - aplica AddForce nessa direção (ForceMode.Impulse = de uma vez só);
    ///    - aplica AddTorque num eixo perpendicular, para o objeto rolar e girar;
    ///    - cria a partícula de impacto no ponto mais próximo do objeto.
    /// 4. Toca o som do pulso, se houver.
    /// </remarks>
    public int FireKineticPulse()
    {
        Vector3 pulseCenter = transform.position + transform.forward * pulseDistance;
        Collider[] hits = Physics.OverlapSphere(pulseCenter, pulseRadius);
        int affectedCount = 0;

        foreach (Collider hit in hits)
        {
            Rigidbody targetRb = hit.attachedRigidbody;

            if (hit.CompareTag("HeavyDebris") && targetRb != null)
            {
                Vector3 direction = (hit.transform.position - transform.position).normalized;
                direction = (direction + Vector3.up * 0.3f).normalized;

                targetRb.AddForce(direction * pulseForce, ForceMode.Impulse);

                Vector3 torqueAxis = Vector3.Cross(Vector3.up, direction);
                targetRb.AddTorque(torqueAxis * pulseTorque, ForceMode.Impulse);

                SpawnImpactEffect(hit.ClosestPoint(transform.position));
                affectedCount++;
            }
        }

        if (pulseSound != null)
        {
            AudioSource.PlayClipAtPoint(pulseSound, transform.position);
        }

        return affectedCount;
    }

    /// <summary>
    /// Cria a partícula de impacto (pedrinhas) no ponto atingido.
    /// </summary>
    /// <param name="position">Ponto do mundo onde a partícula deve aparecer.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Se nenhum prefab de partícula foi definido no Inspector, não faz nada.
    /// Caso contrário, instancia o prefab na posição e o destrói depois de 2
    /// segundos, para não acumular objetos na cena.
    /// </remarks>
    void SpawnImpactEffect(Vector3 position)
```

```
    {
        if (impactParticlePrefab == null)
        {
            return;
        }

        GameObject effect = Instantiate(impactParticlePrefab, position, Quaternion.identity);
        Destroy(effect, 2f);
    }

    /// <summary>
    /// Desenha no editor a esfera da área do pulso (só aparece na aba Scene).
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Ajuda a visualizar e ajustar pulseRadius e pulseDistance no Inspector.
    /// Não aparece no jogo, só com o Player selecionado na aba Scene.
    /// </remarks>
    void OnDrawGizmosSelected()
    {
        Gizmos.color = Color.cyan;
        Gizmos.DrawWireSphere(transform.position + transform.forward * pulseDistance,
pulseRadius);
    }
}
```

6.4 Script: SpawnManager.cs

Cria os Bio-Filtros e as sucatas a partir de vetores de prefabs, sorteando pontos de spawn com `Random.Range` sem repetição. *Autor: Thiago.*

```
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// Cria os Bio-Filtros e as sucatas em posições sorteadas no início da fase.
/// Usa vetores de prefabs e de pontos de spawn, sorteando com Random.Range
/// sem repetir o mesmo ponto, nem entre filtros e sucatas.
/// </summary>
public class SpawnManager : MonoBehaviour
{
    public GameObject[] bioFilterPrefabs;   // vetor de prefabs dos Bio-Filtros
    public Transform[] filterSpawnPoints;   // pontos onde os filtros podem aparecer
    public int filtersToSpawn = 5;          // quantos filtros criar

    public GameObject[] scrapPrefabs;       // vetor de prefabs das sucatas (opcional)
    public Transform[] scrapSpawnPoints;    // pontos onde as sucatas podem aparecer
    public int scrapToSpawn = 8;            // quantas sucatas criar

    private HashSet<Transform> usedPoints = new HashSet<Transform>(); // pontos já ocupados nesta
fase

    /// <summary>
    /// Cria os filtros e as sucatas quando a fase começa.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Chama SpawnObjects duas vezes: primeiro para os Bio-Filtros, depois para as
    /// sucatas. Os pontos usados pelos filtros ficam marcados como ocupados, então
    /// as sucatas só são sorteadas entre os pontos que sobraram. Como o sorteio
    /// acontece a cada vez que a cena carrega, reiniciar a fase muda as posições.
    /// </remarks>
    void Start()
    {
```

```
        int filters = SpawnObjects(bioFilterPrefabs, filterSpawnPoints, filtersToSpawn);
        int scraps = SpawnObjects(scrapPrefabs, scrapSpawnPoints, scrapToSpawn);
        Debug.Log("SpawnManager: " + filters + " filtros e " + scraps + " sucatas criados.");
    }

    /// <summary>
    /// Sorteia pontos sem repetir e cria um prefab sorteado em cada um.
    /// </summary>
    /// <param name="prefabs">Vetor de prefabs que podem ser criados.</param>
    /// <param name="points">Vetor de pontos de spawn disponíveis.</param>
    /// <param name="amount">Quantidade de objetos a criar.</param>
    /// <returns>int: quantos objetos foram realmente criados.</returns>
    /// <remarks>
    /// 1. Se não houver prefabs ou pontos, não cria nada e retorna 0.
    /// 2. Monta a lista de pontos livres: todos os pontos do vetor, menos os que
    ///    já foram ocupados antes (por exemplo, pelos filtros).
    /// 3. Repete até criar a quantidade pedida (ou acabarem os pontos):
    ///    - sorteia um ponto da lista com Random.Range;
    ///    - sorteia qual prefab do vetor será criado;
    ///    - cria o objeto com Instantiate na posição do ponto;
    ///    - remove o ponto da lista e marca como ocupado, para dois objetos
    ///      não nascerem no mesmo lugar.
    /// </remarks>
    int SpawnObjects(GameObject[] prefabs, Transform[] points, int amount)
    {
        if (prefabs == null || prefabs.Length == 0 || points == null || points.Length == 0)
        {
            return 0;
        }

        List<Transform> availablePoints = new List<Transform>();
        foreach (Transform point in points)
        {
            if (point != null && !usedPoints.Contains(point))
            {
                availablePoints.Add(point);
            }
        }

        int created = 0;

        while (created < amount && availablePoints.Count > 0)
        {
            int pointIndex = Random.Range(0, availablePoints.Count);
            int prefabIndex = Random.Range(0, prefabs.Length);

            Transform chosenPoint = availablePoints[pointIndex];
            Instantiate(prefabs[prefabIndex], chosenPoint.position,
prefabs[prefabIndex].transform.rotation);

            availablePoints.RemoveAt(pointIndex);
            usedPoints.Add(chosenPoint);
            created++;
        }

        if (created < amount)
        {
            Debug.LogWarning("SpawnManager: pontos insuficientes. Pedidos " + amount + ", criados
" + created + ".");
        }

        return created;
    }
}
```

6.5 Script: Rotator.cs

Gira continuamente os itens coletáveis no próprio eixo. *Autor: Thiago.*

```
using UnityEngine;

/// <summary>
/// Faz o objeto girar continuamente no próprio eixo vertical.
/// Usado nos Bio-Filtros e nas sucatas para chamar a atenção do jogador.
/// </summary>
public class Rotator : MonoBehaviour
{
    public float rotationSpeed = 90f; // graus por segundo

    /// <summary>
    /// Gira o objeto um pouco a cada frame.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Usa Transform.Rotate em torno do eixo Y (Vector3.up). Multiplicar por
    /// Time.deltaTime faz a velocidade ser a mesma em qualquer computador.
    /// </remarks>
    void Update()
    {
        transform.Rotate(Vector3.up * rotationSpeed * Time.deltaTime);
    }
}
```

6.6 Script: PlayerInventory.cs

Coleta de Bio-Filtros e sucatas por trigger, identificando cada item pela tag. *Autor: Thiago.*

```
using UnityEngine;

/// <summary>
/// Guarda o que o Scrap-E coletou: Bio-Filtros (meta da fase) e sucatas (pontos).
/// A coleta acontece ao encostar nos objetos, identificados pela tag.
/// </summary>
public class PlayerInventory : MonoBehaviour
{
    public int filtersCollected = 0;   // Bio-Filtros que o jogador está carregando
    public int filtersRequired = 5;    // quantos são necessários para ativar a torre
    public int scrapCollected = 0;     // sucatas coletadas (cada uma vale pontos)
    public AudioClip collectSound;     // som de coleta (opcional)

    /// <summary>
    /// Detecta quando o jogador encosta num objeto coletável.
    /// </summary>
    /// <param name="other">Collider do objeto que entrou na área do jogador.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Os coletáveis têm collider marcado como Is Trigger, então o jogador passa
    /// através deles. Pela tag, o script decide o que fazer:
    /// - EcoItem: soma um Bio-Filtro e remove o objeto da cena;
    /// - ScrapCoin: soma uma sucata e remove o objeto da cena.
    /// </remarks>
    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("EcoItem"))
        {
            filtersCollected++;
            Collect(other.gameObject);
            Debug.Log("Bio-Filtros: " + filtersCollected + " / " + filtersRequired);
```

```
        }
        else if (other.CompareTag("ScrapCoin"))
        {
            scrapCollected++;
            Collect(other.gameObject);
            Debug.Log("Sucata: " + scrapCollected);
        }
    }

    /// <summary>
    /// Toca o som de coleta e remove o objeto coletado da cena.
    /// </summary>
    /// <param name="item">Objeto que foi coletado.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// O som é tocado na posição do item com PlayClipAtPoint, que continua
    /// tocando mesmo depois que o item é destruído.
    /// </remarks>
    void Collect(GameObject item)
    {
        if (collectSound != null)
        {
            AudioSource.PlayClipAtPoint(collectSound, item.transform.position);
        }

        Destroy(item);
    }

    /// <summary>
    /// Informa se o jogador já tem todos os Bio-Filtros necessários.
    /// </summary>
    /// <returns>bool: true se filtersCollected chegou em filtersRequired.</returns>
    /// <remarks>
    /// Será usado pela Torre Purificadora para decidir se pode ser ativada.
    /// </remarks>
    public bool HasAllFilters()
    {
        return filtersCollected >= filtersRequired;
    }
}
```

6.7 Script: PlayerHealth.cs

Integridade estrutural (HP): dano contínuo do lodo tóxico, dano de detritos em movimento e Game Over. *Autor: Thiago.*

```
using UnityEngine;

/// <summary>
/// Controla a Integridade Estrutural (HP) do Scrap-E: dano do lodo tóxico,
/// dano de detritos em queda e o Game Over quando o HP chega a zero.
/// </summary>
public class PlayerHealth : MonoBehaviour
{
    public float maxHealth = 100f;
    public float currentHealth = 100f;
    public float slimeDamagePerSecond = 10f;  // dano contínuo enquanto estiver no lodo
    public float debrisDamage = 25f;          // dano de um detrito caindo em cima
    public float debrisMinSpeed = 5f;         // velocidade mínima do impacto para causar dano
    public GameObject deathEffectPrefab;      // faíscas/fumaça de curto-circuito (opcional)

    private bool isDead = false;
    private Collider bodyCollider;          // corpo do jogador (cápsula)
    private string lastDamageSource = "";  // última causa de dano registrada no Console
```

```
    private float lastDamageLogTime = -10f;

    /// <summary>
    /// Começa a fase com o HP cheio e guarda o collider do corpo do jogador.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Copia maxHealth para currentHealth e pega o collider (a cápsula),
    /// usado para conferir se o jogador está mesmo encostando no lodo.
    /// </remarks>
    void Start()
    {
        currentHealth = maxHealth;
        bodyCollider = GetComponent<Collider>();
    }

    /// <summary>
    /// Aplica o dano contínuo enquanto o jogador está dentro de uma poça de lodo.
    /// </summary>
    /// <param name="other">Collider da área em que o jogador está.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// OnTriggerStay roda a cada passo de física enquanto o jogador estiver
    /// dentro do trigger. Se a área tem a tag ToxicSlime, tira um pedaço do
    /// dano por segundo, proporcional ao tempo do passo (Time.fixedDeltaTime).
    /// Por segurança, antes confere se as caixas (bounds) do corpo do jogador e
    /// da poça realmente se cruzam. Se a Unity avisar por engano que o jogador
    /// ainda está na poça depois de sair, o dano é ignorado.
    /// </remarks>
    void OnTriggerStay(Collider other)
    {
        if (!other.CompareTag("ToxicSlime"))
        {
            return;
        }

        if (bodyCollider != null && !bodyCollider.bounds.Intersects(other.bounds))
        {
            return;
        }

        TakeDamage(slimeDamagePerSecond * Time.fixedDeltaTime, "lodo tóxico (" + other.name +
")");
    }

    /// <summary>
    /// Detecta quando um detrito pesado bate no jogador com força.
    /// </summary>
    /// <param name="collision">Dados da colisão, incluindo a velocidade do impacto.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Só causa dano se o objeto tem a tag HeavyDebris e o PRÓPRIO detrito está
    /// se movendo rápido (velocidade do Rigidbody dele acima de debrisMinSpeed).
    /// Assim, encostar numa pedra parada, empurrar um caixote ou cair em cima
    /// de um entulho depois de um pulo não machuca; só um detrito caindo ou
    /// voando na direção do jogador machuca.
    /// </remarks>
    void OnCollisionEnter(Collision collision)
    {
        Rigidbody debrisRb = collision.rigidbody;

        if (collision.gameObject.CompareTag("HeavyDebris") && debrisRb != null &&
debrisRb.linearVelocity.magnitude > debrisMinSpeed)
        {
            TakeDamage(debrisDamage, "detrito (" + collision.gameObject.name + ")");
        }
    }
```

```
    /// <summary>
    /// Tira HP do jogador e verifica se ele foi destruído.
    /// </summary>
    /// <param name="amount">Quantidade de HP a remover.</param>
    /// <param name="source">Descrição do que causou o dano (aparece no Console).</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Ignora se o jogador já morreu.
    /// 2. Registra a causa no Console (no máximo uma vez por segundo para a
    ///    mesma causa, para não lotar o Console com o dano contínuo do lodo).
    /// 3. Subtrai o dano e limita o HP entre 0 e o máximo (Mathf.Clamp).
    /// 4. Se chegou a 0, chama Die().
    /// </remarks>
    public void TakeDamage(float amount, string source = "desconhecido")
    {
        if (isDead)
        {
            return;
        }

        if (source != lastDamageSource || Time.time - lastDamageLogTime > 1f)
        {
            Debug.Log("Dano recebido de " + source + ". HP: " + Mathf.CeilToInt(currentHealth -
amount));
            lastDamageSource = source;
            lastDamageLogTime = Time.time;
        }

        currentHealth = Mathf.Clamp(currentHealth - amount, 0f, maxHealth);

        if (currentHealth <= 0f)
        {
            Die();
        }
    }

    /// <summary>
    /// Recupera HP do jogador (usado pelo cheat E-C-O, se for implementado).
    /// </summary>
    /// <param name="amount">Quantidade de HP a recuperar.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Soma e limita ao máximo. Não faz nada se o jogador já morreu.</remarks>
    public void Heal(float amount)
    {
        if (isDead)
        {
            return;
        }

        currentHealth = Mathf.Clamp(currentHealth + amount, 0f, maxHealth);
    }

    /// <summary>
    /// Destrói o Scrap-E: mostra o curto-circuito e avisa o GameManager.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Marca o jogador como morto, cria o efeito de faíscas (se houver) na
    /// posição dele e chama GameManager.Instance.GameOver().
    /// </remarks>
    void Die()
    {
        isDead = true;

        if (deathEffectPrefab != null)
        {
            Instantiate(deathEffectPrefab, transform.position, Quaternion.identity);
        }
```

```
        if (GameManager.Instance != null)
        {
            GameManager.Instance.GameOver();
        }
    }
}
```

6.8 Script: PurifierTower.cs

Torre Purificadora: verifica as condições da fase, dispara a explosão de partículas e o som, dissipa a névoa e declara a vitória. *Autor: Thiago.*

```
using UnityEngine;

/// <summary>
/// Torre Purificadora: quando o jogador está no console com os 5 Bio-Filtros
/// e aperta E, a torre é ativada, o cenário é purificado e a fase é vencida.
/// Deve ficar num objeto com collider marcado como Is Trigger (a área do console).
/// </summary>
public class PurifierTower : MonoBehaviour
{
    public GameObject purificationEffectPrefab; // explosão de partículas verdes (prefab do
Arthur, opcional)
    public Transform effectPoint;               // onde a explosão aparece (se vazio, usa a
própria torre)
    public AudioClip activationSound;           // som da água / ativação (opcional)
    public GameObject[] blockingDebris;         // entulhos que precisam sair do canal (opcional)
    public Collider channelArea;                // área do canal que precisa ficar livre
(opcional)
    public bool disableFogOnVictory = true;     // a névoa some quando a torre liga

    private bool playerInRange = false;
    private bool isPurified = false;
    private PlayerInventory playerInventory;

    /// <summary>
    /// Detecta quando o jogador entra na área do console.
    /// </summary>
    /// <param name="other">Collider que entrou na área.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Se for o jogador (tag Player), guarda o inventário dele e marca que
    /// está no alcance. Mostra no Console o que falta para ativar.
    /// </remarks>
    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            playerInRange = true;
            playerInventory = other.GetComponent<PlayerInventory>();
            Debug.Log("Torre: aperte E para ativar. " + GetMissingRequirement());
        }
    }

    /// <summary>
    /// Detecta quando o jogador sai da área do console.
    /// </summary>
    /// <param name="other">Collider que saiu da área.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Se for o jogador, marca que ele não está mais no alcance.</remarks>
    void OnTriggerExit(Collider other)
    {
        if (other.CompareTag("Player"))
```

```
        {
            playerInRange = false;
        }
    }

    /// <summary>
    /// Verifica a cada frame se o jogador apertou E perto do console.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Só tenta ativar se o jogador estiver no alcance e a torre ainda não foi ativada.
</remarks>
    void Update()
    {
        if (playerInRange && !isPurified && Input.GetKeyDown(KeyCode.E))
        {
            TryActivate();
        }
    }

    /// <summary>
    /// Tenta ativar a torre, conferindo se todas as condições foram cumpridas.
    /// </summary>
    /// <returns>bool: true se a torre foi ativada; false se ainda falta algo.</returns>
    /// <remarks>
    /// Usa GetMissingRequirement(). Se não falta nada, chama Activate().
    /// Se falta algo, mostra no Console o que é.
    /// </remarks>
    public bool TryActivate()
    {
        string missing = GetMissingRequirement();

        if (missing != "")
        {
            Debug.Log("Torre: ainda não. " + missing);
            return false;
        }

        Activate();
        return true;
    }

    /// <summary>
    /// Verifica o que ainda impede a ativação da torre.
    /// </summary>
    /// <returns>string: descrição do que falta, ou "" se está tudo pronto.</returns>
    /// <remarks>
    /// 1. O jogador precisa ter todos os Bio-Filtros (PlayerInventory.HasAllFilters).
    /// 2. Se channelArea e blockingDebris foram definidos, cada entulho precisa
    ///    estar fora da área do canal (bounds.Contains verifica se um ponto está
    ///    dentro da caixa do collider). Se não foram definidos, essa regra é ignorada.
    /// </remarks>
    string GetMissingRequirement()
    {
        if (playerInventory == null || !playerInventory.HasAllFilters())
        {
            int have = playerInventory != null ? playerInventory.filtersCollected : 0;
            int need = playerInventory != null ? playerInventory.filtersRequired : 5;
            return "Faltam Bio-Filtros (" + have + " / " + need + ").";
        }

        if (channelArea != null)
        {
            int stillBlocking = 0;
            foreach (GameObject debris in blockingDebris)
            {
                if (debris != null && channelArea.bounds.Contains(debris.transform.position))
                {
                    stillBlocking++;
```

```
                }
            }

            if (stillBlocking > 0)
            {
                return "Ainda há " + stillBlocking + " entulho(s) bloqueando o canal.";
            }
        }

        return "";
    }

    /// <summary>
    /// Ativa a torre: efeitos de purificação e vitória.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Marca a torre como purificada (não ativa duas vezes).
    /// 2. Cria a explosão de partículas no effectPoint (ou na torre).
    /// 3. Toca o som de ativação.
    /// 4. Desliga a névoa da cena (RenderSettings.fog), mostrando o céu limpo.
    /// 5. Avisa o GameManager da vitória.
    /// </remarks>
    void Activate()
    {
        isPurified = true;
        Vector3 position = effectPoint != null ? effectPoint.position : transform.position;

        if (purificationEffectPrefab != null)
        {
            Instantiate(purificationEffectPrefab, position, Quaternion.identity);
        }

        if (activationSound != null)
        {
            AudioSource.PlayClipAtPoint(activationSound, position);
        }

        if (disableFogOnVictory)
        {
            RenderSettings.fog = false;
        }

        Debug.Log("Torre Purificadora ativada! O Vale Estéril foi purificado.");

        if (GameManager.Instance != null)
        {
            GameManager.Instance.Victory();
        }
    }
}
```

6.9 Script: GameManager.cs

Estado da fase: pausa, Eco-Timer, vitória, derrota e cálculo da pontuação. *Autor: Thiago.*

```
using UnityEngine;

/// <summary>
/// Controla o estado geral da fase: pausa, vitória, derrota, Eco-Timer e pontuação.
/// Existe um único GameManager por cena, acessível por GameManager.Instance.
/// </summary>
public class GameManager : MonoBehaviour
{
    public static GameManager Instance;   // acesso fácil a partir de outros scripts
```

```
    public PlayerInventory playerInventory;  // inventário do jogador (para a pontuação)
    public MonoBehaviour[] playerControls;   // scripts desligados quando o jogo para
(PlayerMovement, InteractionController)
    public UIManager uiManager;              // interface (HUD e painéis)

    public float ecoTimerSeconds = 180f;     // 3 minutos, como no GDD
    public float victoryScreenDelay = 2f;    // tempo para ver a explosão de partículas antes da
tela de vitória

    public int scorePerScrap = 100;
    public int scoreForCompletion = 1000;
    public int scorePerSecondLeft = 10;

    private bool isPaused = false;
    private bool isGameEnded = false;
    private bool useEcoTimer = false;
    private float timeLeft;
    private float elapsedTime = 0f;

    /// <summary>
    /// Registra este objeto como o GameManager da cena.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Guarda a referência em Instance para que a torre e o HP do jogador
    /// consigam chamar Victory() e GameOver() sem precisar procurar o objeto.
    /// </remarks>
    void Awake()
    {
        Instance = this;
    }

    /// <summary>
    /// Prepara a fase: libera o tempo e liga o Eco-Timer se ele foi ativado no menu.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Garante que o tempo do jogo está correndo (Time.timeScale = 1).
    /// 2. Lê a escolha feita no Menu Principal (MainMenuController.useEcoTimer).
    /// 3. Se o Eco-Timer estiver ligado, começa a contagem e mostra o cronômetro.
    /// </remarks>
    void Start()
    {
        Time.timeScale = 1f;
        useEcoTimer = MainMenuController.useEcoTimer;
        timeLeft = ecoTimerSeconds;

        if (uiManager != null)
        {
            uiManager.ShowTimer(useEcoTimer);
        }
    }

    /// <summary>
    /// A cada frame: verifica a tecla de pausa e atualiza os cronômetros.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Se a fase já terminou, não faz nada.
    /// 2. Esc liga e desliga a pausa.
    /// 3. Se não está pausado, soma o tempo jogado e, no modo Eco-Timer,
    ///    desconta o tempo restante. Se ele zerar, é Game Over.
    /// </remarks>
    void Update()
    {
        if (isGameEnded)
        {
```

```
            return;
        }

        if (Input.GetKeyDown(KeyCode.Escape))
        {
            TogglePause();
        }

        if (isPaused)
        {
            return;
        }

        elapsedTime += Time.deltaTime;

        if (useEcoTimer)
        {
            timeLeft -= Time.deltaTime;

            if (uiManager != null)
            {
                uiManager.UpdateTimer(timeLeft);
            }

            if (timeLeft <= 0f)
            {
                timeLeft = 0f;
                GameOver();
            }
        }
    }

    /// <summary>
    /// Pausa ou despausa o jogo.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Inverte o estado de pausa, congela ou libera o tempo (Time.timeScale),
    /// liga ou desliga os controles do jogador e mostra ou esconde o painel de pausa.
    /// Também é chamado pelo botão Continuar.
    /// </remarks>
    public void TogglePause()
    {
        if (isGameEnded)
        {
            return;
        }

        isPaused = !isPaused;
        Time.timeScale = isPaused ? 0f : 1f;
        SetPlayerControls(!isPaused);

        if (uiManager != null)
        {
            uiManager.ShowPause(isPaused);
        }
    }

    /// <summary>
    /// Encerra a fase com vitória. Chamado pela Torre Purificadora.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Ignora se a fase já terminou (evita vitória dupla).
    /// 2. Desliga os controles do jogador.
    /// 3. Espera alguns segundos (victoryScreenDelay) para o jogador ver a
    ///    transformação do cenário e então mostra a tela de vitória.
    /// </remarks>
```

```
    public void Victory()
    {
        if (isGameEnded)
        {
            return;
        }

        isGameEnded = true;
        SetPlayerControls(false);
        Invoke(nameof(ShowVictoryScreen), victoryScreenDelay);
    }

    /// <summary>
    /// Mostra a tela de vitória com a pontuação final e congela o jogo.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Calcula a pontuação, envia para o UIManager junto com o tempo jogado
    /// e para o tempo do jogo (Time.timeScale = 0).
    /// </remarks>
    void ShowVictoryScreen()
    {
        if (uiManager != null)
        {
            uiManager.ShowVictory(CalculateScore(), elapsedTime);
        }

        Time.timeScale = 0f;
    }

    /// <summary>
    /// Encerra a fase com derrota. Chamado pelo HP do jogador ou pelo Eco-Timer.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Ignora se a fase já terminou. Caso contrário, desliga os controles,
    /// mostra a tela de Game Over e congela o jogo.
    /// </remarks>
    public void GameOver()
    {
        if (isGameEnded)
        {
            return;
        }

        isGameEnded = true;
        SetPlayerControls(false);

        if (uiManager != null)
        {
            uiManager.ShowGameOver();
        }

        Time.timeScale = 0f;
    }

    /// <summary>
    /// Calcula a pontuação final da fase.
    /// </summary>
    /// <returns>int: pontos da sucata + bônus de conclusão + bônus de tempo (só no Eco-Timer).
</returns>
    /// <remarks>
    /// Cada sucata vale scorePerScrap (100). Concluir vale scoreForCompletion (1000).
    /// No modo Eco-Timer, cada segundo restante vale scorePerSecondLeft (10).
    /// </remarks>
    public int CalculateScore()
    {
        int score = scoreForCompletion;
```

```
        if (playerInventory != null)
        {
            score += playerInventory.scrapCollected * scorePerScrap;
        }

        if (useEcoTimer)
        {
            score += Mathf.CeilToInt(timeLeft) * scorePerSecondLeft;
        }

        return score;
    }

    /// <summary>
    /// Liga ou desliga os scripts de controle do jogador.
    /// </summary>
    /// <param name="enabled">true para o jogador poder se mover; false para travar.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Percorre o vetor playerControls (normalmente PlayerMovement e
    /// InteractionController) e muda o enabled de cada um. Um script
    /// desligado não roda o Update, então o jogador não anda nem dispara o pulso.
    /// </remarks>
    void SetPlayerControls(bool enabled)
    {
        foreach (MonoBehaviour control in playerControls)
        {
            if (control != null)
            {
                control.enabled = enabled;
            }
        }
    }
}
```

6.10 Script: UIManager.cs

Atualiza o HUD (HP, filtros, sucata, cronômetro) e exibe os painéis de pausa, vitória e Game Over. *Autor: Thiago.*

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Atualiza o HUD do Rafael (HP, filtros, sucata, cronômetro) e mostra os painéis
/// de pausa, vitória e game over. Fica no objeto HUD_Canvas e encontra sozinho
/// os elementos pelos nomes combinados no contexto do projeto.
/// </summary>
public class UIManager : MonoBehaviour
{
    public PlayerInventory playerInventory;  // de onde vêm os números de filtros e sucata
    public PlayerHealth playerHealth;        // de onde vem o HP

    private Slider barraHP;
    private Text textoFiltros;
    private Text textoSucata;
    private Text textoTimer;
    private Text textoPontuacao;
    private GameObject painelPausa;
    private GameObject painelVitoria;
    private GameObject painelGameOver;

    /// <summary>
```

```
    /// Encontra os elementos do HUD pelos nomes e liga o botão Continuar.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Usa transform.Find com os nomes do contrato (BarraHP, TextoFiltros, ...).
    ///    Assim não é preciso arrastar cada elemento no Inspector.
    /// 2. Liga o BotaoContinuar ao TogglePause do GameManager.
    /// 3. Garante que os painéis começam escondidos.
    /// </remarks>
    void Awake()
    {
        barraHP = FindComponent<Slider>("BarraHP");
        textoFiltros = FindComponent<Text>("TextoFiltros");
        textoSucata = FindComponent<Text>("TextoSucata");
        textoTimer = FindComponent<Text>("TextoTimer");
        textoPontuacao = FindComponent<Text>("PainelVitoria/TextoPontuacao");

        painelPausa = FindObject("PainelPausa");
        painelVitoria = FindObject("PainelVitoria");
        painelGameOver = FindObject("PainelGameOver");

        Button botaoContinuar = FindComponent<Button>("PainelPausa/BotaoContinuar");
        if (botaoContinuar != null)
        {
            botaoContinuar.onClick.AddListener(OnContinueClicked);
        }

        SetActive(painelPausa, false);
        SetActive(painelVitoria, false);
        SetActive(painelGameOver, false);
    }

    /// <summary>
    /// Atualiza os números do HUD a cada frame.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// Lê os valores direto do inventário e do HP do jogador e escreve nos
    /// textos e na barra. Se alguma referência estiver vazia, pula aquela parte.
    /// </remarks>
    void Update()
    {
        if (playerInventory != null)
        {
            if (textoFiltros != null)
            {
                textoFiltros.text = "Bio-Filtros: " + playerInventory.filtersCollected + " / " +
playerInventory.filtersRequired;
            }

            if (textoSucata != null)
            {
                textoSucata.text = "Sucata: " + playerInventory.scrapCollected;
            }
        }

        if (playerHealth != null && barraHP != null)
        {
            barraHP.maxValue = playerHealth.maxHealth;
            barraHP.value = playerHealth.currentHealth;
        }
    }

    /// <summary>
    /// Mostra ou esconde o cronômetro do Eco-Timer.
    /// </summary>
    /// <param name="show">true para mostrar; false para esconder.</param>
    /// <returns>void (sem retorno).</returns>
```

```
    /// <remarks>Chamado pelo GameManager no início da fase.</remarks>
    public void ShowTimer(bool show)
    {
        if (textoTimer != null)
        {
            textoTimer.gameObject.SetActive(show);
        }
    }

    /// <summary>
    /// Escreve o tempo restante do Eco-Timer no formato minutos:segundos.
    /// </summary>
    /// <param name="secondsLeft">Segundos que ainda faltam.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Arredonda para cima, para mostrar 0:01 até o último instante.</remarks>
    public void UpdateTimer(float secondsLeft)
    {
        if (textoTimer != null)
        {
            textoTimer.text = FormatTime(Mathf.Max(0f, secondsLeft));
        }
    }

    /// <summary>
    /// Mostra ou esconde o painel de pausa.
    /// </summary>
    /// <param name="show">true para mostrar; false para esconder.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Chamado pelo GameManager ao apertar Esc ou Continuar.</remarks>
    public void ShowPause(bool show)
    {
        SetActive(painelPausa, show);
    }

    /// <summary>
    /// Mostra o painel de vitória com a pontuação e o tempo de jogo.
    /// </summary>
    /// <param name="score">Pontuação final calculada pelo GameManager.</param>
    /// <param name="elapsedSeconds">Tempo total jogado, em segundos.</param>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Escreve no TextoPontuacao e ativa o PainelVitoria.</remarks>
    public void ShowVictory(int score, float elapsedSeconds)
    {
        if (textoPontuacao != null)
        {
            textoPontuacao.text = "Pontuação: " + score + "\nTempo: " +
FormatTime(elapsedSeconds);
        }

        SetActive(painelVitoria, true);
    }

    /// <summary>
    /// Mostra o painel de Game Over.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Os botões Reiniciar e Menu já foram ligados pelo Rafael ao MainMenuController.
</remarks>
    public void ShowGameOver()
    {
        SetActive(painelGameOver, true);
    }

    /// <summary>
    /// Reação ao clique no botão Continuar do painel de pausa.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Pede ao GameManager para despausar o jogo.</remarks>
```

```
    void OnContinueClicked()
    {
        if (GameManager.Instance != null)
        {
            GameManager.Instance.TogglePause();
        }
    }

    /// <summary>
    /// Converte segundos em texto no formato m:ss.
    /// </summary>
    /// <param name="seconds">Quantidade de segundos.</param>
    /// <returns>string: o tempo formatado, por exemplo "2:05".</returns>
    /// <remarks>Arredonda para cima e separa minutos e segundos com divisão inteira e resto.
</remarks>
    string FormatTime(float seconds)
    {
        int total = Mathf.CeilToInt(seconds);
        int minutes = total / 60;
        int secs = total % 60;
        return minutes + ":" + secs.ToString("00");
    }

    /// <summary>
    /// Procura um filho do HUD pelo caminho e devolve um componente dele.
    /// </summary>
    /// <param name="path">Caminho do objeto dentro do HUD_Canvas (ex.:
"PainelPausa/BotaoContinuar").</param>
    /// <returns>O componente encontrado, ou null se o objeto não existir.</returns>
    /// <remarks>Se não encontrar, avisa no Console qual nome está faltando.</remarks>
    T FindComponent<T>(string path) where T : Component
    {
        Transform child = transform.Find(path);
        if (child == null)
        {
            Debug.LogWarning("UIManager: não encontrei '" + path + "' dentro do " +
gameObject.name);
            return null;
        }

        return child.GetComponent<T>();
    }

    /// <summary>
    /// Procura um filho do HUD pelo caminho e devolve o GameObject.
    /// </summary>
    /// <param name="path">Caminho do objeto dentro do HUD_Canvas.</param>
    /// <returns>O GameObject encontrado, ou null se não existir.</returns>
    /// <remarks>Se não encontrar, avisa no Console qual nome está faltando.</remarks>
    GameObject FindObject(string path)
    {
        Transform child = transform.Find(path);
        if (child == null)
        {
            Debug.LogWarning("UIManager: não encontrei '" + path + "' dentro do " +
gameObject.name);
            return null;
        }

        return child.gameObject;
    }

    /// <summary>
    /// Ativa ou desativa um objeto, se ele existir.
    /// </summary>
    /// <param name="target">Objeto a ser ativado ou desativado.</param>
    /// <param name="active">true para ativar; false para desativar.</param>
    /// <returns>void (sem retorno).</returns>
```

```
    /// <remarks>Evita erro caso algum painel não tenha sido encontrado.</remarks>
    void SetActive(GameObject target, bool active)
    {
        if (target != null)
        {
            target.SetActive(active);
        }
    }
}
```

6.11 Script: MainMenuController.cs

Menu Principal, vitrine 3D do Scrap-E e botões de navegação, incluindo o botão obrigatório de reiniciar o jogo. *Autor: Rafael.*

```
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

/// <summary>
/// Controla o Menu Principal, a vitrine 3D do Scrap-E e os botões de navegação
/// de telas (Iniciar, Reiniciar, Menu Principal, Controles, Eco-Timer e Sair).
/// </summary>
public class MainMenuController : MonoBehaviour
{
    public static bool useEcoTimer = false;

    public string nomeCenaJogo = "Nivel1";
    public string nomeCenaMenu = "MenuPrincipal";
    public GameObject painelControles;
    public Text textoStatusEcoTimer;
    public Transform roboVitrine;
    public float velocidadeGiroVitrine = 25f;

    /// <summary>
    /// Inicializa o estado do menu quando a cena começa, destravando o tempo
    /// e atualizando o texto do modo Eco-Timer se houver referência configurada.
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
    /// <remarks>
    /// 1. Garante que Time.timeScale volte para 1f caso o jogador tenha vindo de uma tela
pausada.
    /// 2. Se textoStatusEcoTimer estiver atribuído no Inspector, atualiza o rótulo com o estado
atual de useEcoTimer.
    /// </remarks>
    void Start()
    {
        Time.timeScale = 1f;

        if (textoStatusEcoTimer != null)
        {
            textoStatusEcoTimer.text = useEcoTimer ? "ECO-TIMER: ATIVADO (3 MIN)" : "ECO-TIMER:
DESATIVADO";
        }
    }

    /// <summary>
    /// Gira continuamente o robô da vitrine 3D no menu e verifica o atalho de reinício rápido
(tecla R).
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
    /// <remarks>
```

```
    /// 1. Se roboVitrine estiver atribuído no Inspector, aplica Transform.Rotate no eixo Y
(Vector3.up) a cada frame.
    /// 2. Se o jogador pressionar a tecla R fora do MenuPrincipal, recarrega a cena atual
imediatamente.
    /// </remarks>
    void Update()
    {
        if (roboVitrine != null)
        {
            roboVitrine.Rotate(Vector3.up * velocidadeGiroVitrine * Time.deltaTime);
        }

        if (Input.GetKeyDown(KeyCode.R) && SceneManager.GetActiveScene().name != nomeCenaMenu)
        {
            ReiniciarJogo();
        }
    }

    /// <summary>
    /// Inicia a partida carregando a cena principal do jogo (Nivel1).
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
    /// <remarks>
    /// 1. Garante que Time.timeScale esteja em 1f.
    /// 2. Chama SceneManager.LoadScene passando o nome configurado em nomeCenaJogo.
    /// </remarks>
    public void IniciarJogo()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(nomeCenaJogo);
    }

    /// <summary>
    /// Retorna para a cena do Menu Principal a partir de qualquer tela.
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
    /// <remarks>
    /// 1. Restaura Time.timeScale para 1f para não manter o jogo congelado.
    /// 2. Carrega a cena definida em nomeCenaMenu ("MenuPrincipal").
    /// </remarks>
    public void CarregarMenuPrincipal()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(nomeCenaMenu);
    }

    /// <summary>
    /// Reinicia a cena atual do zero, restaurando o estado inicial da fase.
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
    /// <remarks>
    /// 1. Restaura Time.timeScale para 1f.
    /// 2. Obtém o nome da cena ativa com SceneManager.GetActiveScene().name e a recarrega.
    /// </remarks>
    public void ReiniciarJogo()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }

    /// <summary>
    /// Abre ou fecha o painel de controles e instruções no Menu Principal.
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
```

```
    /// <remarks>
    /// 1. Verifica se painelControles foi atribuído no Inspector.
    /// 2. Inverte o estado ativo do GameObject usando SetActive(!painelControles.activeSelf).
    /// </remarks>
    public void AlternarPainelControles()
    {
        if (painelControles != null)
        {
            painelControles.SetActive(!painelControles.activeSelf);
        }
    }

    /// <summary>
    /// Liga ou desliga a opção de desafio Eco-Timer (limite de 3 minutos) e atualiza o texto na
interface.
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
    /// <remarks>
    /// 1. Inverte o valor booleano estático useEcoTimer.
    /// 2. Se textoStatusEcoTimer estiver atribuído, atualiza o texto indicando se o Eco-Timer
está ativado ou desativado.
    /// </remarks>
    public void AlternarEcoTimer()
    {
        useEcoTimer = !useEcoTimer;

        if (textoStatusEcoTimer != null)
        {
            textoStatusEcoTimer.text = useEcoTimer ? "ECO-TIMER: ATIVADO (3 MIN)" : "ECO-TIMER:
DESATIVADO";
        }
    }

    /// <summary>
    /// Encerra a execução do jogo (ou registra mensagem no Console quando testado no Editor).
    /// </summary>
    /// <param name="nenhum">Esta função não recebe parâmetros.</param>
    /// <returns>void, sem retorno.</returns>
    /// <remarks>
    /// 1. Exibe mensagem no Console com Debug.Log confirmando o acionamento do botão Sair.
    /// 2. Chama Application.Quit() para fechar o executável do jogo.
    /// </remarks>
    public void SairDoJogo()
    {
        Debug.Log("Saindo do Eco-Scrapper: Reclamation...");
        Application.Quit();
    }
}
```

6.12 Script: CheatCode.cs

Trapaça de desenvolvedor E-C-O (GDD 3.4): recupera a vida e dobra a velocidade por alguns segundos, para facilitar a demonstração em bancada. *Autor: Thiago.*

```
using System.Collections;
using UnityEngine;

/// <summary>
/// Trapaça de desenvolvedor (GDD 3.4): digitar E, C, O em sequência recupera
/// toda a vida do Scrap-E e dobra a velocidade por alguns segundos.
/// Serve para facilitar a demonstração do jogo em bancada.
/// </summary>
public class CheatCode : MonoBehaviour
```

```
{
    public float maxTimeBetweenKeys = 1.5f;  // tempo máximo entre uma tecla e a próxima
    public float boostMultiplier = 2f;       // quanto a velocidade é multiplicada
    public float boostDuration = 5f;         // quanto tempo o impulso dura (segundos)
    public AudioClip cheatSound;             // som de confirmação (opcional)

    private KeyCode[] sequence = { KeyCode.E, KeyCode.C, KeyCode.O };
    private int currentIndex = 0;            // quantas teclas da sequência já foram acertadas
    private float lastKeyTime = 0f;

    private PlayerHealth playerHealth;
    private PlayerMovement playerMovement;
    private float originalSpeed;
    private float originalRunSpeed;
    private bool isBoosting = false;
    private Coroutine boostRoutine;

    /// <summary>
    /// Pega os scripts de vida e de movimento do próprio jogador.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>Os dois precisam estar no mesmo objeto (o Player).</remarks>
    void Start()
    {
        playerHealth = GetComponent<PlayerHealth>();
        playerMovement = GetComponent<PlayerMovement>();
    }

    /// <summary>
    /// Acompanha as teclas digitadas e verifica se formam a sequência E-C-O.
    /// </summary>
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Se passou tempo demais desde a última tecla certa, recomeça a sequência.
    /// 2. Se a próxima tecla esperada foi apertada, avança uma posição.
    ///    Ao completar as três, ativa a trapaça e recomeça.
    /// 3. Se apertou uma das letras do código fora de ordem, recomeça
    ///    (e aceita essa tecla como início, caso seja o E).
    /// </remarks>
    void Update()
    {
        if (currentIndex > 0 && Time.time - lastKeyTime > maxTimeBetweenKeys)
        {
            currentIndex = 0;
        }

        if (Input.GetKeyDown(sequence[currentIndex]))
        {
            currentIndex++;
            lastKeyTime = Time.time;

            if (currentIndex >= sequence.Length)
            {
                currentIndex = 0;
                ActivateCheat();
            }
        }
        else if (Input.GetKeyDown(KeyCode.E) || Input.GetKeyDown(KeyCode.C) ||
Input.GetKeyDown(KeyCode.O))
        {
            currentIndex = Input.GetKeyDown(KeyCode.E) ? 1 : 0;
            lastKeyTime = Time.time;
        }
    }

    /// <summary>
    /// Aplica os efeitos da trapaça: vida cheia e impulso de velocidade.
    /// </summary>
```

```
    /// <returns>void (sem retorno).</returns>
    /// <remarks>
    /// 1. Recupera toda a vida com PlayerHealth.Heal.
    /// 2. Inicia (ou reinicia) a corrotina do impulso de velocidade.
    /// 3. Toca o som de confirmação e registra no Console.
    /// </remarks>
    void ActivateCheat()
    {
        if (playerHealth != null)
        {
            playerHealth.Heal(playerHealth.maxHealth);
        }

        if (playerMovement != null)
        {
            if (boostRoutine != null)
            {
                StopCoroutine(boostRoutine);
            }

            boostRoutine = StartCoroutine(SpeedBoost());
        }

        if (cheatSound != null)
        {
            AudioSource.PlayClipAtPoint(cheatSound, transform.position);
        }

        Debug.Log("Cheat E-C-O ativado: vida cheia e velocidade x" + boostMultiplier + " por " +
boostDuration + "s.");
    }

    /// <summary>
    /// Dobra a velocidade do jogador por um tempo e depois volta ao normal.
    /// </summary>
    /// <returns>IEnumerator: corrotina que a Unity executa ao longo de vários frames.</returns>
    /// <remarks>
    /// 1. Na primeira ativação, guarda as velocidades originais (se o cheat for
    ///    usado de novo durante o impulso, não multiplica em cima do já multiplicado).
    /// 2. Multiplica speed e runSpeed do PlayerMovement por boostMultiplier.
    /// 3. Espera boostDuration segundos (WaitForSeconds).
    /// 4. Restaura as velocidades originais.
    /// </remarks>
    IEnumerator SpeedBoost()
    {
        if (!isBoosting)
        {
            originalSpeed = playerMovement.speed;
            originalRunSpeed = playerMovement.runSpeed;
            isBoosting = true;
        }

        playerMovement.speed = originalSpeed * boostMultiplier;
        playerMovement.runSpeed = originalRunSpeed * boostMultiplier;

        yield return new WaitForSeconds(boostDuration);

        playerMovement.speed = originalSpeed;
        playerMovement.runSpeed = originalRunSpeed;
        isBoosting = false;
        boostRoutine = null;
    }
}
```

7. Inteligência Artificial

7.1 IA Oponente e Inimiga

- O Nível 1 **não possui inimigos com IA**. O foco do 1º bimestre é a exploração e os puzzles físicos, e o desafio vem do ambiente (lodo tóxico, desmoronamentos e o Eco-Timer opcional), conforme descrito na seção 3.6.
- Planejado para o 2º bimestre: Drones de Vigilância Corrompidos no Nível 3 (Pântano Sintético). Eles patrulham rotas fixas usando NavMesh e, ao detectar o Scrap-E dentro do seu raio de visão, passam a persegui-lo; o jogador precisa desviar ou se esconder atrás de estruturas para escapar.

7.2 Personagens Não Jogáveis (NPCs)

- **ARIA (NPC de Suporte):** não possui corpo físico no cenário e se comunica pelo rádio do HUD. Suas falas são disparadas por eventos do jogo:
  - Início do nível: diálogo de abertura (seção 2.1).
  - Primeiro Bio-Filtro coletado: confirmação e lembrete de quantos faltam.
  - Aproximação de uma poça de lodo: aviso de perigo.
  - Todos os 5 filtros coletados: indicação para seguir até a Torre Purificadora.
  - Vitória: confirmação da purificação do Vale Estéril.
- **Fauna Nativa (NPCs Ambientais):** Sand-Gecko, Canyon Swift, Eco-Hare, Besouro-Blindado e Peixe Cintilante. São ativados somente após a purificação e seguem um comportamento simples:
  - Ocioso: permanece parado executando sua animação.
  - Vagando: desloca-se entre pontos pré-definidos do cenário.
  - Fuga: quando o Scrap-E se aproxima a menos de alguns metros, o animal se afasta, reforçando a sensação de vida no cenário.

7.3 IA de Suporte (Detecção e Navegação)

- **Detecção de Colisão:** feita pelo motor de física da Unity (PhysX), com Colliders e Rigidbodies. Os eventos de colisão e de trigger identificam o tipo de objeto pela Tag (`EcoItem`, `ScrapCoin`, `ToxicSlime`, `HeavyDebris`, `PurifierTrigger`) e disparam a reação correta: coletar, causar dano ou permitir interação.
- **Detecção do Jogador:** volumes de trigger posicionados no cenário reconhecem a Tag `Player` para exibir os prompts de ajuda (seção 5.2), disparar as falas da ARIA e ativar a fuga da fauna.
- **Detecção de Alvos do Pulso Cinético:** uma esfera de detecção (`Physics.OverlapSphere`) à frente do Scrap-E encontra os detritos dentro do alcance do pulso.
- **Descoberta de Caminho (Pathfinding):** não é utilizada no Nível 1. No 2º bimestre, o NavMesh será usado para a fauna e para os drones inimigos encontrarem rotas pelo terreno.

8. Requisitos Técnicos de Execução

- Game Engine: Unity 6 (Versão exata: 6000.0.8f1 - compatível com os laboratórios acadêmicos).
- Template de Renderização: Universal 3D (Universal Render Pipeline - URP).
- Hardware Alvo: PCs do laboratório com arquitetura x86_64, Windows 10/11, com suporte gráfico a DirectX 11/12 e Vulkan.
- Entrada e Periféricos: Suporte exclusivo a Teclado e Mouse.
- Requisitos de Rede: Nulo (experiência local single-player offline).
- Pipeline de Assets Externos: Modelagem Low-Poly via Blender / Asset Store, animações humanoides via Mixamo, e texturas estilizadas com shaders URP leves.

9. Arte do Jogo

9.1 Estilo Pretendido

- 3D Low-Poly estilizado: formas simples e facetadas, poucas texturas detalhadas e cores marcantes. O estilo garante boa leitura visual dos objetos interativos e bom desempenho nos computadores do laboratório.
- Paleta em dois estados, reforçando o tema de Ecorenovação:
  - Antes da purificação: ocre, cinza e marrom enferrujado, com névoa densa e iluminação fria e apagada.
  - Depois da purificação: verdes vivos, azul da água limpa e luz dourada, com o céu aberto.
- Elementos interativos com destaque visual: Bio-Filtros com brilho emissivo verde e rotação contínua; sucatas coletáveis com brilho metálico; lodo tóxico com textura emissiva verde-amarelada.

9.2 Assets Chave e Como Serão Desenvolvidos

Asset	Origem e Desenvolvimento
Terreno do Vale Estéril	Modelado na ferramenta Terrain da própria Unity (relevo, ravina, leito do rio e platôs), pintado com camadas de textura de solo árido, cascalho, rocha e lodo.
Scrap-E (protagonista)	Modelo humanoide low-poly (Asset Store ou free3d), com rig compatível com Mixamo para as animações Idle, Walk, Run, Jump e Fall.
Rochas, flora e sucatas	Pacotes low-poly gratuitos da Unity Asset Store e do free3d, organizados nas categorias da seção 4.1.

Torre Purificadora e console	Montados a partir de peças industriais da Asset Store combinadas na própria Unity, com materiais emissivos para indicar o estado (desligada / ativa).
Texturas e materiais	Texturas do TextureBox e dos próprios pacotes, aplicadas com materiais URP Lit.
Partículas	Criadas no Shuriken Particle System da Unity: fumaça, pedrinhas de impacto e onda de purificação (seção 3.2).
Áudio	Efeitos sonoros e trilhas ambientes de bibliotecas gratuitas, separados entre fase contaminada e fase purificada (seção 5.4).

Todos os assets de terceiros serão validados com o professor antes de serem utilizados, conforme orientação da disciplina.